PROJECT

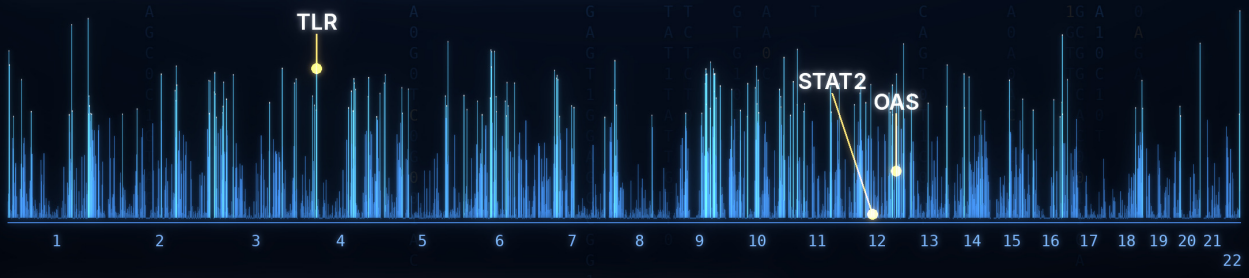
HUMAN EVOLUTION

Hinweise auf Neandertaler-Erbe im Immun-Genom

Big-Data-Pipeline mit Spark, PostgreSQL, MongoDB, Neo4j und Airflow



KANDIDATEN-ALLELE JE 1-MB-FENSTER, 22 CHROMOSOMEN (DATEN DES PROJEKTS)



Heiko Han | Lehrgang AI Engineer, Modul 5 Big Data Engineering | Oktober 2026

Project Human Evolution

Neandertaler-Kandidaten-Allele in Immun-Genregionen heutiger Menschen – eine Big-Data-Pipeline mit PostgreSQL, MongoDB, Neo4j, Python-ETL, Airflow und Spark

Abschlussprojekt, Lehrgang AI Engineer, Modul 5 Big Data Engineering | Autor: Heiko Han | Stand: 21.10.2026

Transparenzhinweis: *Das Projekt wurde als Team von Heiko Han und dem KI-Assistenten Claude (Anthropic) erarbeitet. Die Fragestellung, die Auswahl der Daten, die Entscheidungen und die Ausführung aller Läufe stammen von Heiko Han; Claude hat Code, Texte und Auswertungen vorgeschlagen und gemeinsam mit ihm geprüft. Aussagen, die nicht belegt sind, sind im Text als „nicht geprüft“ gekennzeichnet.*

1. Zusammenfassung und Fragestellung

Das Projekt untersucht, in welchen Regionen des heutigen menschlichen Genoms Varianten vorkommen, die auch in archaischen Genomen (Neandertaler, Denisova-Mensch) die abgeleitete Base tragen, in Afrikanern aber selten sind. Solche Positionen sind **Hinweise auf mögliches Neandertaler-Erbe**, kein Beweis. Als Pilotregionen dienen drei Immun-Genregionen: OAS1/2/3 (Chromosom 12), TLR1/6/10 (Chromosom 4) und STAT2 (Chromosom 12). Der OAS-Locus gilt in der Literatur als bekanntes Neandertaler-Erbe und dient als Gegenprobe für die Pipeline. Zusätzlich wurde dieselbe Auswertung für alle 22 Autosomen durchgeführt.

Eckdaten: 61.599.150 biallelische SNVs der 1000-Genomes-Daten (3.202 Personen, 26 Populationen) wurden mit dem Ahnenzustand und vier archaischen Genomen verknüpft. Es ergeben sich **88.632 Kandidaten-Allele** (0,14 % aller SNVs), davon 81 in den Pilotregionen (OAS 51, TLR 30, STAT2 0). Die Ergebnisse liegen konsistent in PostgreSQL, MongoDB und Neo4j vor und werden in einem einzelnen, offline lauffähigen HTML-Dashboard dargestellt.

Was das Projekt nicht aussagt: Es bestimmt keinen genomweiten Neandertaler-Anteil einer Person oder Population. Die Zahlen beschreiben Kandidaten-Allele in Regionen und deren Häufigkeit.

2. Daten und Quellen

Rohdaten (ca. 80 GB komprimiert) sind nicht Teil der Abgabe, sondern über die folgenden Quellen frei erhältlich. Arbeitsbuild ist GRCh38 (hg38); Ust'-Ishim liegt in hg19 vor und wird per Umrechnung (UCSC-Chain-Datei, pyliftover) nach hg38 übertragen.

Rolle	Datensatz	Build	Link / Lizenz
Heutiger Mensch	1000 Genomes, NYGC 30x, 3.202 Personen, phasierte VCFs Chr 1–22 (ca. 29 GB)	hg38	internationalgenome.org (Ankündigung); Daten: ftp.1000genomes.ebi.ac.uk/vol1/ftp/data_collections/1000G_2504_high_coverage/working/20220422_3202_phased_SNV_INDEL_SV/
Neandertaler, Denisova	Skov et al., „Archaic variants from Altai, Vindija, Chagyrskaya and Denisova (hg38)“, BCF, Chr 1–22 (ca. 940 MB)	hg38	zenodo.org/records/13368126, DOI 10.5281/zenodo.13368126, CC BY 4.0
Ahnenzustand	Ensembl Ancestral Alleles, Release 112 (abgeleitet aus dem 10-Primaten-EPO-Alignment inkl. Schimpanse)	GRCh38	ftp.ensembl.org/pub/release-112/fasta/ancestral_alleles/ (Datei homo_sapiens_ancestor_GRCh38.tar.gz)
Früher Mensch	Ust'-Ishim (ca. 45.000 Jahre), VCF je Chromosom (ca. 45 GB); nur in den Pilotregionen ausgewertet	hg19	cdna.eva.mpg.de/neandertal/Vindija/VCF/Ust_Ishim/ (Beschreibung: …/Vindija/VCF/README)
Gene (Koordinaten)	NCBI GRCh38.p14, Annotation GTF (GCF_000001405.40)	GRCh38	ncbi.nlm.nih.gov/datasets/genome/GCF_000001405.40/
Umrechnung	UCSC hg19ToHg38.over.chain.gz und hg38ToHg19.over.chain.gz	–	hgdownload.soe.ucsc.edu/goldenPath/hg19/liftOver/ bzw. …/hg38/liftOver/

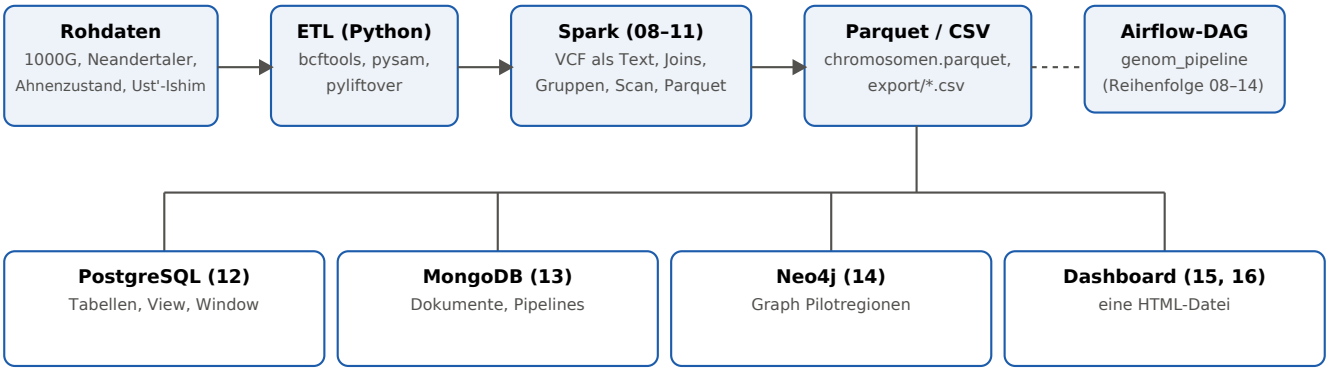
Populationstabelle: ftp.1000genomes.ebi.ac.uk/vol1/ftp/data_collections/1000G_2504_high_coverage/20130606_g1k_3202_samples_ped_population.txt (5 Großgruppen AFR 893, AMR 490, EAS 585, EUR 633, SAS 601 Personen; 26 Populationen). Die Lizenz- bzw. Nutzungsbedingungen der Quellen außer Zenodo (CC BY 4.0) wurden nicht einzeln geprüft; die Daten werden nicht weitergegeben.

2.1 Nicht verwendete Daten und Gründe

Was	Grund
AADR (v66, 1240k)	nur ca. 1,23 Mio. SNPs genomweit, für kleine Regionen zu dünn; überwiegend pseudohaploid; hg19; in der geladenen Version fehlte die .geno-Datei
neanderthal.hg38.filt.vcf (GATK)	Herkunft nicht dokumentiert; mit GATK statt snpAD erzeugt (nicht schadensbewusst für alte DNA); nur Variantenpositionen, keine Referenzpositionen
MPI-Vindija-VCFs (hg19)	der Zenodo-Datensatz enthält die Aufrufe bereits in hg38, mit Referenzpositionen und fehlenden Genotypen
Schimpansen-Genom direkt	Ensembl liefert den Ahnenzustand fertig in menschlichen Koordinaten; ein eigenes Cross-Species-Alignment wäre aufwendig und fehleranfällig
HLA-Region	extrem polymorph, an einer Referenz schwer sauber auszuwerten
Cloud (GCS, Dataproc, Hail, Colab)	nicht Kursinhalt (Spark lief im Kurs lokal); Daten müssten verschoben werden; knappe Zeit
MLlib-Klassifikation	würde bei diesem Thema künstlich wirken
Alter Prototyp	nur Testdaten; Maskenfehler, 1,72 Mio. exakte Duplikate bei 2,11 Mio. Zeilen, OAS2/OAS3 fehlten; nicht weiterverwendet

3. Architektur und Lehrgangsbezug

Die Verarbeitung ist eine Kette aus nummerierten Python-Skripten (00–16) und einem Airflow-DAG. Rohdaten werden per ETL in ein einheitliches Format gebracht, mit Spark in Parquet-Tabellen ausgewertet und die Ergebnisse in drei Datenbanken und ein Dashboard überführt.



Lehrgangsthema	Einsatz im Projekt
PostgreSQL (Tag 1–6)	Skript 12: Datenbank genom mit Tabellen regionen, gene, populationen, kandidaten (88.632), haeufigkeiten (2.304.432), scan_1mb; Primär- und Fremdschlüssel nach dem Laden; JOIN über Position, LEFT JOIN (STAT2 ohne Kandidaten), Window Function (Rang je Chromosom), Aggregationen, View v_kandidat_mittel
MongoDB (Tag 7–8)	Skript 13: ein Dokument je Kandidat mit eingebetteter Liste der 26 Populationshäufigkeiten und archaischen Genotypen; vier Indizes; vier Aggregation Pipelines
Neo4j (Tag 9–10)	Skript 14: Graph für die Pilotregionen (81 Varianten, 7 Gene, 3 Regionen, 26 Populationen, 5 Großgruppen, 4 archaische Individuen); vier Cypher-Abfragen; alle Knoten mit zusätzlichem Label :Genom
ETL mit Python (Tag 11–12)	bcftools-Extraktion BCF → TSV (22 Dateien), Vereinheitlichung der Chromosomennamen und Koordinaten (hg19 → hg38), Ahnenzustand aus FASTA, Prüfungen und Fehlerbehandlung, Wiederholbarkeit der Läufe

Lehrgangsthema	Einsatz im Projekt
Airflow (Tag 13–14)	DAG genom_pipeline: BashOperator-Kette 08 → 09 → 10 → 11 → 12 → 13 → 14, schedule=None, Zeitlimits je Task
Spark (Tag 15–20)	Skripte 08–11: 1000-Genomes-VCFs als Text einlesen (regexp), Parquet partitioniert nach Chromosom, DataFrames, Spark SQL, Joins über Datenquellen, Aggregationen (Kandidaten je Megabase und je Population), Window Functions; local[4], 4 GB Treiberspeicher. MLlib (PCA) war optional und ist nicht umgesetzt

Der Airflow-DAG wurde mit airflow tasks test für den Task neo4j_14 geprüft (Exit-Code 0). Die Spark-Tasks wurden nicht über den DAG, sondern direkt als Skripte ausgeführt; ein Airflow-Webserver/Scheduler wurde nicht eingerichtet.

4. Methode

4.1 Pipeline im Überblick

- **Nur biallelische SNVs** aus den 1000 Genomes; Indels, Strukturvarianten und Multiallele sind ausgeschlossen.
- **Ahnenzustand** je Position aus der Ensembl-Datei (Großbuchstaben ACGT sicher, Kleinbuchstaben unsicher, N unbekannt). Die abgeleitete Base ist die Base, die nicht dem Ahnenzustand entspricht. Entspricht sie der Referenz (ref), bedeutet der Genotyp 0/0 „abgeleitet“; dafür gibt es die eigene Umrechnung n_abgeleitet.
- **Archaische Genome** (Altai, Vindija, Chagyrskaya, Denisova): Genotypen je Position; nicht abgedeckte Positionen fehlen (./.).
- **Häufigkeiten**: je Großgruppe aus den INFO-Feldern der 2.504 Unverwandten; je Einzelpopulation (26) aus den Genotypen der 2.590 Gründer (Mittel über Populationen, nicht nach Personenzahl gewichtet).
- **Ust'-Ishim** nur in den Pilotregionen (Filter DP ≥ 10, GQ ≥ 30).

4.2 Einteilung in Gruppen (eigene Festlegung)

Jede Position erhält genau eine Gruppe, geprüft in dieser Reihenfolge: (1) *unklar_Ahnenzustand*, wenn kein Ahnenallel bestimmbar ist; (2) *archaisch_nicht_abgedeckt*, wenn kein archaisches Individuum die Position abdeckt; (3) **geteilt_Neandertaler_nicht_afrikanisch** (Kandidat), wenn mindestens eines von Altai, Vindija, Chagyrskaya die abgeleitete Base trägt **und** die abgeleitete Häufigkeit in AFR ≤ 3 % liegt **und** das Mittel der Großgruppen AMR, EAS, EUR, SAS ≥ 5 % beträgt; (4) *geteilt_Neandertaler_sonst*; (5) *nur_Denisova_geteilt*; (6) *nur_Mensch*. Die Schwellenwerte 3 % und 5 % sowie DP 10 und GQ 30 sind eigene, nicht aus der Literatur übernommene Festlegungen; eine Sensitivitätsanalyse wurde nicht durchgeführt.

Gruppe	SNVs genomweit	Anteil
nur_Mensch	31.054.733	50,41 %
archaisch_nicht_abgedeckt	23.451.869	38,07 %
unklar_Ahnenzustand	4.621.455	7,50 %
geteilt_Neandertaler_sonst	1.960.618	3,18 %
nur_Denisova_geteilt	421.843	0,68 %
geteilt_Neandertaler_nicht_afrikanisch (Kandidaten)	88.632	0,14 %
gesamt	**61.599.150**	100 %

Validierung: Die Pipeline für ganze Chromosomen reproduziert im OAS-Fenster exakt die Zahlen der zuvor separat gerechneten Pilotregion (745 / 51 / 50 / 60 / 1.065 / 18).

5. Ergebnisse

5.1 Pilotregionen

Gruppe	OAS	STAT2	TLR
nur_Mensch	1.065	122	1.039
archaisch_nicht_abgedeckt	745	124	1.125
nur_Denisova_geteilt	60	2	14
Kandidaten	**51**	**0**	**30**
geteilt_Neandertaler_sonst	50	4	100
unklar_Ahnenzustand	18	6	83
SNVs gesamt	**1.989**	**258**	**2.391**

- **OAS (Gegenprobe bestanden):** 51 Kandidaten, verteilt über 112.907.362–113.011.311 (Chr 12). Denisova trägt bei keinem OAS-Kandidaten die abgeleitete Base. Die zehn häufigsten OAS1-Varianten mit allen drei Neandertalern haben identische mittlere EUR-Häufigkeit (0,365): ein gemeinsamer Haplotyp-Block, keine zehn unabhängigen Belege.
- **TLR:** 30 Kandidaten. Denisova trägt die abgeleitete Base bei TLR1 an 6 von 11, TLR10 an 3 von 6, TLR6 an 1 von 7 Kandidaten (Zuordnung nur für Varianten innerhalb der Genkoordinaten).
- **STAT2:** keine Kandidaten (negativer Befund). Die 1000 Genomes enthalten keine melanesischen Populationen; ob ein archaisches Signal dort zu erwarten wäre, wurde nicht geprüft.
- **Ust'-Ishim** trägt bei allen abgedeckten Kandidaten (OAS 49 von 51, TLR 30 von 30) die Ahnenbase, also nicht die Neandertaler-Variante. Ein einzelnes Individuum (zwei Haplotypen) belegt weder das eine noch das andere; die Beobachtung wird nur berichtet.

5.2 Häufigkeit in den Populationen

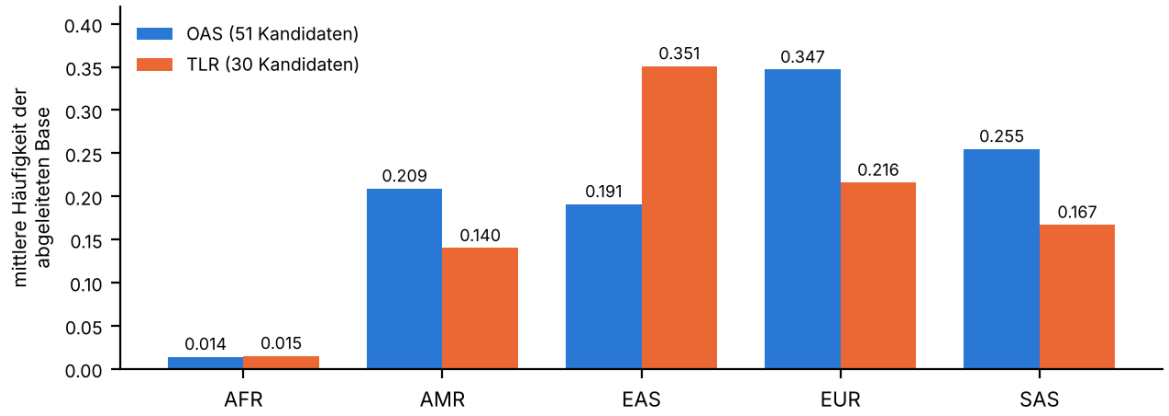


Abbildung: mittlere Häufigkeit der abgeleiteten Base der Kandidaten je Großgruppe (Mittel über Populationen; identisch in PostgreSQL, MongoDB und Neo4j). Kein genomweiter Vermischungsanteil.

Über alle 88.632 Kandidaten liegt die mittlere Häufigkeit in Ostasien bei 11,2–11,6 %, in Amerika bei 8,7–11,8 % (PEL 11,8 %), in Europa bei 9,2–10,0 %, in Südasien bei 8,6–9,0 % und in Afrika bei 0,3–2,6 % (ASW 2,6 %, ACB 1,4 %; diese beiden Populationen haben nach allgemeiner Kenntnis europäischen Anteil, nicht geprüft). Das Muster – niedrig in Afrika, höher außerhalb – entspricht der Erwartung für Introgression, folgt aber auch aus der Gruppendefinition selbst (AFR ≤ 3 % ist Bedingung) und ist deshalb kein unabhängiger Beleg.

5.3 Genomweit

- **Stärke des Hinweises:** Von den 88.632 Kandidaten tragen 67.992 die abgeleitete Base in allen drei Neandertaler-Individuen, 11.868 in zwei und 8.772 in einem. Denisova trägt sie bei 22.401 (ca. 25 %); ob das Introgression oder älteren gemeinsamen Polymorphismus anzeigt, lässt sich mit diesen Daten nicht trennen.

- **Verteilung:** 0,6 bis 2,4 Kandidaten je 1000 SNV und Chromosom (chr12 2,41; chr9 2,20; chr10 2,12; chr17, 19, 21 unter 0,7). Die 2.777 Megabase-Fenster zeigen Spitzen von 10 bis 17 je 1000 SNV (z. B. chr22 bei 49 Mb mit 373 Kandidaten). Ursachen (Rekombination, Abdeckung, Selektion) wurden nicht untersucht.
- **Chromosom 12 mit Spark:** 2.879.549 SNVs in 238 Sekunden (local[4], 4 GB); Gesamtlauf über alle 22 Chromosomen siehe Skript 08.

5.4 Gegenprobe über drei Datenbanken

Dieselben Kandidaten und Häufigkeiten wurden unabhängig in PostgreSQL (SQL-Aggregation), MongoDB (Aggregation Pipeline) und Neo4j (Cypher) ausgewertet. Die Mittelwerte stimmen überein (OAS 51 Kandidaten: AFR 0,014 / AMR 0,209 / EAS 0,191 / EUR 0,347 / SAS 0,255; TLR 30 Kandidaten: 0,015 / 0,140 / 0,351 / 0,216 / 0,167). Ein erster MongoDB-Lauf lieferte 47 bzw. 24 statt 51 bzw. 30 Kandidaten, weil die Region dort zunächst nur über die Genkörper bestimmt wurde; nach Umstellung auf die Pilotfenster aus der Konfiguration stimmten alle Zahlen überein.

5.5 Dashboard

Eine einzelne HTML-Datei (ca. 257 KB, offline, ohne externe Bibliotheken) mit Kennzahlen, Sunburst-Diagramm der Gruppen (Bereich Genom/OAS/TLR/STAT2), Weltkarte der 26 Populationen (Punktpositionen von Hand gesetzt, Weltumriss Natural Earth, Public Domain), Genomscan je Megabase mit markierten Pilotregionen, Stärke des Hinweises, Histogramm der Häufigkeit bei Nicht-Afrikanern, Kandidatentabelle (381 Zeilen: alle 81 Pilotkandidaten plus die 300 mit der höchsten mittleren Häufigkeit bei Nicht-Afrikanern) und einem Abschnitt „Methode und Grenzen“. Erzeugt von den Skripten 15 und 16; zum Anzeigen genügt ein Browser.

6. Grenzen

- 38 % aller Positionen sind archaisch nicht abgedeckt, weitere 7,5 % haben keinen bestimmbaren Ahnenzustand; beide Gruppen können Kandidaten enthalten, die nicht erkannt werden.
- Die Gruppendefinition (AFR ≤ 3 %, Nicht-AFR ≥ 5 %) setzt das erwartete Populationsmuster voraus und ist eine eigene Festlegung; andere Schwellen ergäben andere Zahlen.
- Die abgeleitete Base entspricht oft der hg38-Referenz; ein Einfluss auf die archaischen Aufrufe (Referenzverzerrung) wurde nicht ausgeschlossen.
- Die Aufrufe der archaischen Genome stammen aus alter DNA; Schäden können einzelne Genotypen beeinflussen. Die Qualität der Aufrufe im Datensatz von Skov et al. wurde hier nicht erneut geprüft.
- Nicht erfasst: Indels, Strukturvarianten, Multiallele, Geschlechtschromosomen.
- Benachbarte Kandidaten sind durch Kopplung nicht unabhängig (Haplotyp-Blöcke).
- Häufigkeiten je Population stammen aus den 2.590 Gründern; einzelne Kandidaten liegen deshalb im Dashboard-Mittel unter 5 %, obwohl die Gruppendefinition die Häufigkeit der 2.504 Unverwandten nutzt.
- Die 1000-Genomes-Daten enthalten verwandte Personen (Eltern/Kind-Spalten); der Einfluss auf Häufigkeiten wurde nicht untersucht (die Gruppendefinition nutzt die Unverwandten-Felder).
- Ust'-Ishim ist nur in den Pilotregionen ausgewertet.
- Auffälligkeit in den Quelldaten: Position 112951306 hat REF = ALT (T>T).

Nicht umgesetzt (Ausblick): Schimpansen-Genotyp statt Ahnenzustand, Selektionstests, Alter des Genflusses, statistischer Test der Herkunft einzelner Varianten, genomweiter Neandertaler-Anteil, PCA mit MLlib, Ust'-Ishim genomweit, Flanken um die Genkörper (Regulation).

7. Probleme und Erkenntnisse

- **Downloads:** Der Server der 1000 Genomes lieferte schwankend 0,06 bis 3,5 MB/s; Abbrüche wurden mit Wiederaufnahme (wget --timeout=30 --tries=0) abgefangen.

- **Defekte Datei-Blöcke:** In mehreren heruntergeladenen VCF.gz-Dateien waren einzelne BGZF-Blöcke beschädigt. Sie wurden über eine CRC-Prüfung je Block gefunden und per HTTP-Range-Anfrage gezielt neu geladen. Danach stimmten die Dateigrößen mit den Serverangaben überein; die Skripte überspringen Chromosomen, deren Größe nicht der erwarteten entspricht.
- **Arbeitsspeicher:** Der Laptop (15 GiB RAM, oft nur 6–9 GiB verfügbar) begrenzte Spark auf local[4] mit 4 GB; die Verarbeitung erfolgt je Chromosom und schreibt Parquet-Ordner mit Fertig-Marke (_SUCCESS), fertige Chromosomen werden bei einem Neustart übersprungen.
- **Wiederholbarkeit:** Die Datenbankskripte sind mit der Umgebungsvariablen GENOM_NEU_AUFBAUEN=1 wiederholbar und löschen nur eigene Daten (Schema public der Datenbank genom, Collection genom.kandidaten, Knoten mit Label :Genom).
- **Werkzeug-Versionen:** pandas und neuere pyarrow-Versionen kollidierten; in der Umgebung bde-kurs ist pyarrow==20.0.0 festgelegt.
- **Selbstkontrolle:** Abweichungen zwischen den Datenbanken (siehe 5.4) zeigten einen Fehler in der Regionszuordnung, der durch den Vergleich aufgefallen ist.

8. Reproduzierbarkeit

Umgebung: Linux Mint; Spyder 6.1.5 mit Interpreter der conda-Umgebung **bde-kurs** (Python 3.11, PySpark 3.5.9, OpenJDK 21, bcftools 1.24, htslib 1.23.1, pysam 0.23.3, pyliftover 0.4.1, pandas, pyarrow 20.0.0, psycopg2, pymongo, neo4j-Treiber, sqlalchemy); Airflow 2.9.3 in der getrennten Umgebung **bde-kurs-airflow**. Datenbanken im Docker-Container: PostgreSQL (localhost:5432, Datenbank genom), MongoDB 7.0 (localhost:27017), Neo4j 5.26 (bolt://localhost:7687). Zugangsdaten sind lokale Kursdaten.

Reihenfolge: 08 → 09 → 10 → 11 → 12 → 13 → 14 → 15 → 16, entweder einzeln in Spyder oder über den DAG. Die Erklärung je Skript (Zweck, Eingabe, Ausgabe, Kursbezug) steht in Anhang A. Alle Pfade stehen nur in 00_config.py.

9. Offene und nicht geprüfte Punkte

- **Literatur:** Der OAS-Locus als Neandertaler-Erbe ist belegt durch Sams et al. 2016, „Adaptively introgressed Neandertal haplotype at the OAS locus functionally impacts innate immune responses in humans“, Genome Biology, DOI 10.1186/s13059-016-1098-6 (acht Autoren; Seite geprüft). Weitere Literatur zu Neandertaler-Erbe in TLR1/6/10 und STAT2 wurde nur als Titel gefunden, Autoren und Jahr sind **nicht geprüft** und werden hier deshalb nicht zitiert.
- Abdeckung von Ust'-Ishim (aus dem Gedächtnis ca. 42x, nicht belegt; Lesetiefe an den Kandidaten ca. 17–64).
- Bedeutung der Dateinamen-Filter mq25_mapab100 bei Ust'-Ishim (nicht geprüft).
- Nutzungsbedingungen der Quellen außer Zenodo (CC BY 4.0) nicht einzeln geprüft.
- Ursachen der Spitzen im Genomscan und der Unterschiede zwischen den Chromosomen.

Anhang A: Erklärung der Skripte

Alle Pfade stehen nur in 00_config.py. Die Skripte sind für Spyder geschrieben (# %%-Zellen), laufen mit dem Python der Umgebung bde-kurs und können über den Airflow-DAG gestartet werden. Reihenfolge der Hauptkette: 08, 09, 10, 11, 12, 13, 14, 15, 16. Die Skripte 01 bis 07 sind die Entwicklung und Absicherung an der Pilotregion; sie schreiben bis auf 06 nichts.

Vorverarbeitung ohne Skript: Neandertaler-BCF nach TSV

Dieser ETL-Schritt wurde von Hand mit bcftools ausgeführt und ist in keinem Skript abgelegt. Der Befehl unten ist sinngemäß wiedergegeben; der ursprünglich verwendete Wortlaut wurde nicht aufbewahrt. Ergebnis: 22 Dateien Zwischen/neandertaler/chrN.tsv.gz (zusammen 829 MB, 7 Spalten: Position, Ref, Alt und je ein Genotyp für Altai, Vindija, Denisova, Chagyrskaya; fehlende Genotypen als ./.). Die Spaltenreihenfolge der Genotypen folgt der Probenreihenfolge im BCF und ist in genom_chrom.py fest verdrahtet; sie lässt sich mit bcftools query -l individuals_highcov.12.bcf prüfen.

```
cd Datensätze/Neandertaler_hg38
for c in $(seq 1 22); do
  bcftools query -f '%POS\t%REF\t%ALT[\t%GT]\n' individuals_highcov.$c.bcf \
    | gzip > ../../Zwischen/neandertaler/chr$c.tsv.gz
done
```

Gemeinsame Module

Datei	Zweck	Eingabe	Ausgabe	Kursbezug
`00_config.py`	Zentrale Einstellungen: alle Pfade, die 7 Gene, die 3 Regionen, Probennamen, Funktion für den Pfad der 1000-Genomes-VCF je Chromosom	–	–	ETL: Konfiguration an einer Stelle
`genom_funktionen.py`	Lesefunktionen mit pysam: Neandertaler-BCF, Ahnenzustand (FASTA), 1000-Genomes-Info, Ust'-Ishim (hg19 nach hg38 per pyliftover)	Rohdaten, `00_config`	Pandas-Tabellen	ETL: Extrahieren und Vereinheitlichen
`genom_spark.py`	Spark-Pipeline für eine Region: Varianten, Ahnenzustand, archaische Genotypen, Ust'-Ishim, Gruppe	Pandas-Tabellen aus `genom_funktionen`	Spark-DataFrame	Spark: DataFrames, Joins
`genom_chrom.py`	Spark-Pipeline für ganze Chromosomen: VCF als Text einlesen, Ahnenzustand über Zeilennummer und Versatz in der FASTA, Neandertaler-TSV, Gruppeneinteilung (ohne Ust'-Ishim)	1000-Genomes-VCF, Ensembl-FASTA, `Zwischen/neandertaler/chrN.tsv.gz`	Spark-DataFrame	Spark: Text einlesen, Joins, Spark SQL

Entwicklung an der Pilotregion (01 bis 07)

Skript	Zweck	Eingabe	Ausgabe	Kursbezug
`01_neandertaler_oas.py`	OAS-Cluster: Neandertaler-Genotypen lesen und an Spark übergeben	Neandertaler-BCF	nur Konsolenausgabe	ETL, Spark

Skript	Zweck	Eingabe	Ausgabe	Kursbezug
02_oas_neandertaler_ahnenzustand.py	OAS: Neandertaler plus Ahnenzustand, Koordinatenkontrolle	wie 01, Ahnenzustand	nur Konsolenausgabe	ETL
03_oas_haeufigkeiten.py	OAS: Neandertaler plus Ahnenzustand plus 1000-Genomes-Häufigkeiten (Spark-Join)	wie 02, 1000-Genomes-VCF	nur Konsolenausgabe	Spark: Joins
04_oas_gruppen.py	OAS: Varianten in Gruppen einteilen, Ausgangspunkt sind die 1000-Genomes-Varianten	wie 03	nur Konsolenausgabe	Spark: Aggregationen
05_oas_ust_ishim.py	Wie 04, dazu Ust'-Ishim (DP >= 10, GQ >= 30)	wie 04, Ust'-Ishim-VCF, Chain-Dateien	nur Konsolenausgabe	ETL, Spark
06_pilot_parquet.py	Drei Pilotregionen in einer Tabelle	Funktionen aus genom_spark	Ausgabe/pilot_varianten.parquet (partitioniert nach region, 4.638 Zeilen); bricht ab, wenn das Ziel existiert	Spark: Parquet
07_chromosom_test.py	Test mit Chromosom 12; im OAS-Fenster müssen die Zahlen aus 04 herauskommen	genom_chrom	nur Konsolenausgabe	Spark

Hauptkette (08 bis 16)

Skript	Zweck	Eingabe	Ausgabe	Kursbezug
08_alle_chromosomen.py	Spark über alle 22 Chromosomen; Chromosomen mit unvollständiger VCF oder fertiger Ausgabe werden übersprungen	1000-Genomes-VCF, FASTA, Neandertaler-TSV	Ausgabe/chromosomen.parquet/chrom=chrN	Spark: Parquet, Partitionen, Aggregationen
09_scan_pro_megabase.py	Genomweiter Scan: Gruppen je 1-Mb-Fenster zählen	chromosomen.parquet	Ausgabe/scan_1mb.parquet und scan_1mb.csv	Spark: Aggregationen, Window Functions
10_kandidaten_populationen.py	Häufigkeit der abgeleiteten Base je Kandidat und je Population (26) aus den Genotypen	chromosomen.parquet (Kandidaten), 1000-Genomes-VCF, Populationstabelle	Ausgabe/kandidaten_pop.parquet/chrom=chrN, kandidaten_pop_mittel.csv	Spark, ETL
11_export_fuer_datenbanken.py	Kandidaten und Häufigkeiten als CSV für die Datenbanken	chromosomen.parquet (Kandidaten), kandidaten_pop.parquet	Ausgabe/export/kandidaten.csv, haeufigkeiten.csv	ETL: Laden
12_postgres_genom.py	PostgreSQL: Datenbank genom, Tabellen, View, fünf Abfragen	Ausgabe/export/*.csv, scan_1mb.csv	Datenbank genom (Schema public)	PostgreSQL
13_mongodb_genom.py	MongoDB: ein Dokument je Kandidat mit eingebetteten Häufigkeiten, 4 Indizes, 4 Aggregation Pipelines	Ausgabe/export/*.csv	Collection genom.kandidaten	MongoDB

Skript	Zweck	Eingabe	Ausgabe	Kursbezug
14_neo4j_genom.py	Neo4j: Graph der Pilotregionen (Varianten, Gene, Regionen, Populationen, archaische Individuen), 4 Cypher-Abfragen	Ausgabe/export/*.csv	Knoten mit Label :Genom	Neo4j
15_dashboard_daten.py	Dashboard-Daten berechnen	chromosomen.parquet (Spalten pos, gruppe), scan_1mb.csv, export/*.csv	Ausgabe/dashboard/daten.json	Python/pandas, ETL
16_dashboard_bauen.py	Daten in die HTML-Vorlage einsetzen	Code/dashboard_vorlage.html, daten.json	Ausgabe/dashboard/dashboard.html (eine Datei, offline)	–
genom_pipeline.py	Airflow-DAG, Kette 08 bis 14, schedule=None, Datenbank-Tasks mit GENOM_NEU_AUFBAUEN=1	die Skripte	– (die Skripte schreiben)	Airflow

Hinweise

- Die Datenbankskripte 12 bis 14 löschen nur eigene Daten: Schema public der Datenbank genom (PostgreSQL), Collection genom.kandidaten (MongoDB), Knoten mit Label :Genom (Neo4j). Ein Neuaufbau ist nur mit GENOM_NEU_AUFBAUEN=1 möglich.
- Zugangsdaten in 12 bis 14 sind lokale Kurs-Zugangsdaten und müssen vor einer Veröffentlichung in Umgebungsvariablen ausgelagert werden.
- dashboard_vorlage.html ist die Vorlage für Skript 16 (Gestaltung, JavaScript, Weltkarte).